

Relatório de Auditoria de Segurança

Projeto: FutStats (futebol-analise)

Data da Auditoria: 01 de Setembro de 2026	Tipo de Auditoria: White-box Source Code Review
Escopo Auditado: Repositório Completo (Frontend, API Serverless, Supabase, Configs)	Classificação: Restrito / Confidencial

1. Detecção da Stack & Nota Metodológica

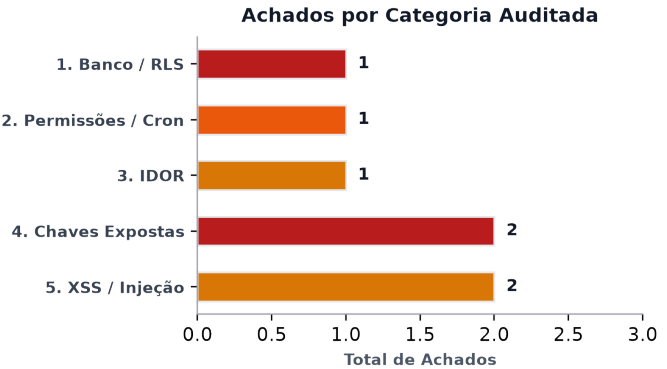
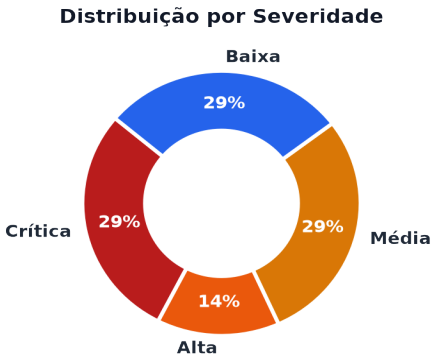
Antes do início dos testes, a arquitetura e stack técnica do projeto foram integralmente mapeadas para contextualizar a análise de vulnerabilidades nas 5 categorias requeridas:

Camada	Tecnologias Identificadas	Mapeamento Metodológico da Auditoria
Frontend	Vanilla JavaScript (ES6+ SPA), Hash Routing (#/), PWA (sw.js), WebPush API.	Inspeção de XSS via innerHTML, URLs javascript:, vazamento de chaves e controle de estado local.
Backend	Node.js Serverless Functions na Vercel (api/football.js, api/news.js, api/img.js, api/cron-alerts.js).	Verificação de controle de acesso em endpoints cron, proxies de API externa, SSRF, injeção de comandos e gestão de segredos.
Edge / BaaS	Supabase PostgreSQL (PostgREST / @supabase/supabase-js), Supabase Edge Functions (Deno).	Auditoria de Row Level Security (RLS) na tabela push_subscriptions e uso da chave anônima/serviço.
Auth & Tenant	Aplicação aberta (sem login/sessão de usuário). Isolamento de preferências via localStorage e push por endpoint.	Mapeamento de IDOR e isolamento de dados no cliente e na persistência remota do Supabase.
Deploy / CI	Vercel Git Integration, .env.local (ignorado no git), sem Docker/Helm/Terraform.	Verificação de segredos commitados, defaults em código e arquivos de backup (backups/).

2. Resumo Executivo

A auditoria identificou um total de **7 achados verificados** distribuídos em diferentes níveis de criticidade. Os pontos mais urgentes residem na **exposição da Chave Privada VAPID e chave da API-Football como fallbacks em código-fonte** e na **ausência de Row Level Security (RLS) restritivo na tabela do Supabase**, permitindo que qualquer cliente consulte ou apague as inscrições push de todos os usuários.

CRÍTICA (2) VAPID Key & RLS Ausente	ALTA (1) Cron Endpoint Aberto	MÉDIA (2) IDOR Push & XSS News	BAIXA (2) Backups & Backend Unescape	PONTOS FORTES (5) Whitelists & Local Escapes
---	---	--	--	--



3. Pontos Fortes e Riscos Centrais

Pontos Fortes (Controles Verificados e Corretos)	Pontos Fracos (Riscos Centrais)
• Whitelist de Endpoints no Proxy: <code>api/football.js:6-27</code> restringe endpoints da API externa via <code>ALLOWED_ENDPOINTS</code> Set, bloqueando SSRF e chamadas arbitrárias. • Validação de Host de Imagens: <code>api/img.js:8-11</code> rejeita URLs fora de <code>https://media.api-sports.io/</code>, mitigando Open Proxy. • Função de Escape HTML: <code>public/app.js:87-94</code> implementa <code>escapeHtml</code> para sanitização de strings em múltiplos templates. • Isolamento de Escalações no Cliente: O montador de escalas opera estritamente no <code>localStorage</code>, sem tráfego de dados sensíveis na nuvem. • Proteção Git: O arquivo <code>.gitignore</code> isola corretamente <code>.env</code> e <code>.env.local</code>.	• Chaves de Produção no Código: Chave privada VAPID e chave da API-Football expostas como fallbacks literais em <code>api/cron-alerts.js</code>. • Tabela Push sem RLS: <code>push_subscriptions</code> no Supabase permite dump ou exclusão anônima de todas as inscrições push cadastradas. • Disparador de Cron sem Autenticação: <code>api/cron-alerts.js</code> pode ser acionado livremente por qualquer agente externo, esgotando quota da API e gerando custos. • XSS via Feed de Notícias: Títulos e links RSS de terceiros inseridos sem escape em <code>loadTeamNews</code>.

4. Tabela de Achados Detalhados por Categoria

Abaixo estão listados todos os achados comprovados no código real, com sua localização precisa e mecanismo de risco:

Sev.	Categoria	Arquivo : Linha	Descrição da Falha & Evidência
CRÍTICA	1. Banco / RLS	public/app.js:80-81, 674-683 api/cron-alerts.js:9-10, 23-25	Tabela push_subscriptions sem RLS restritivo: O backend usa SUPABASE_ANON_KEY para select(*). Como a anon key é pública no frontend, qualquer atacante pode ler ou deletar em massa todos os tokens push de todos os usuários via API PostgREST.
ALTA	2. Permissões / Cron	api/cron-alerts.js:20-320	Endpoint Cron de Disparo Push sem Autenticação: A rota serverless não valida token secreto de cron (CRON_SECRET) nem header do Vercel Cron. Qualquer usuário pode fazer requisições ilimitadas, disparando push em massa e consumindo quota da API-Football.
MÉDIA	3. IDOR	public/app.js:674-683 api/cron-alerts.js:69, 297-300	Manipulação e Sobrescrita de Assinaturas Push por Endpoint: A tabela PostgREST aceita operações de merge/update baseadas apenas no parâmetro endpoint sem validação de posse do cliente ou assinatura de sessão.
CRÍTICA	4. Chaves Expostas	api/cron-alerts.js:8, 10, 13	Chave Privada VAPID e API Key Hardcoded como Fallbacks: Segredo criptográfico privado VAPID (gWKwUrc5XB...) e chave da API-Sports embutidos em código-fonte via operador . Permite falsificação de WebPush e roubo de quota.
BAIXA	4. Chaves Expostas	backups/snapshot_stable/api/cron-alerts.js:8-13	Credenciais Replicadas em Diretório de Backup: Os arquivos estáticos e compactados de snapshot contêm as mesmas chaves confidenciais expostas.
MÉDIA	5. XSS / Injeção	public/app.js:1275-1288	Injeção Direta de Título, Fonte e Link RSS em innerHTML: Dados externos retornados por feeds de notícias são renderizados sem passar por escapeHtml() e links não validam protocolo seguro.
BAIXA	5. XSS / Injeção	api/news.js:203-209	Dessanitização de Entidades HTML no Backend: O parser de RSS converte < e > em caracteres literais < e >, agravando o risco de XSS caso o frontend renderize em innerHTML.

5. Recomendações Priorizadas de Correção

P1 (Imediato / Blocker): Remover todos os fallbacks literais de segredos em api/cron-alerts.js, rotacionar a chave da API-Football e o par de chaves VAPID no provedor, e configurar as variáveis no painel da Vercel.

P1 (Imediato / Blocker): Habilitar Row Level Security (RLS) na tabela push_subscriptions do Supabase. Utilizar a chave de serviço (SUPABASE_SERVICE_ROLE_KEY) exclusivamente no backend e restringir a anon key apenas a inserções anônimas sem permissão de SELECT geral.

P2 (Alta Prioridade): Adicionar autenticação por bearer token em api/cron-alerts.js (process.env.CRON_SECRET) para impedir acionamento externo indiscriminado.

P2 (Alta Prioridade): Aplicar escapeHtml() e validação de URL em loadTeamNews em public/app.js.

P3 (Média Prioridade): Limpar os backups estáticos em backups/ e adicionar script de validação de startup para abortar execução se variáveis críticas estiverem ausentes.

6. Issues para o GitHub (Prontas para Cópia)

Os blocos abaixo contêm o texto Markdown completo, estruturado e pronto para criar as issues no GitHub do projeto:

--- ISSUE 1 ---

Título: [Segurança] Exposição de Chave Privada VAPID e API Key em fallback de código

Labels: security, severidade:crítica, backend

Descrição do Problema

O arquivo `api/cron-alerts.js` define credenciais confidenciais de produção como valores de fallback do operador `||` caso as variáveis de ambiente não estejam populadas. Entre os valores expostos estão a chave privada de assinatura VAPID (`gWKwUrc5XB...`) e a chave de acesso da API-Sports (`a70fc65a67...`).

Evidência de Código

Arquivo: `api/cron-alerts.js:8-13`

```
const FOOTBALL_API_KEY = process.env.FOOTBALL_API_KEY || "a70fc65a67c10981ace9813a509db554";
const VAPID_PRIVATE_KEY = process.env.VAPID_PRIVATE_KEY || "gWKwUrc5XBpYU0BEXM1ha_M3ugoo5JbM7mQSMt4Lk_c";
```

Impacto

Permite a emissão não autorizada de notificações WebPush arbitrárias para dispositivos de usuários inscritos e o consumo/esgotamento da cota de requisições da API de futebol.

Sugestão de Correção

1. Remover imediatamente as strings de fallback do código-fonte.
2. Exigir que a função aborte com erro 500 caso as variáveis não existam no `process.env`.
3. Rotacionar as chaves VAPID e a chave da API-Sports no provedor.

Critérios de Aceite

- Nenhum segredo ou chave privada presente como literal no código-fonte.
- Inicialização aborta com erro explícito se `process.env.VAPID_PRIVATE_KEY` for nula.
- Par de chaves VAPID e API Key rotacionados em produção.

--- FIM ISSUE 1 ---

--- ISSUE 2 ---

Título: [Segurança] Tabela `push_subscriptions` exposta sem Row Level Security (RLS) restritivo

Labels: security, severidade:crítica, database, supabase

Descrição do Problema

A tabela `push_subscriptions` no Supabase está acessível para a role `anon`. Como o backend em `api/cron-alerts.js` usa a chave anônima para ler todos os registros (`select(*)`), a tabela não possui RLS restritivo de leitura. Qualquer usuário de posse da `SUPABASE_ANON_KEY` pública do frontend pode consultar todos os endpoints e preferências ou executar `DELETE` em massa.

Evidência de Código

Arquivo: `public/app.js:80-81, 674-683` e `api/cron-alerts.js:9-10, 23-25`

```
const SUPABASE_KEY = process.env.SUPABASE_ANON_KEY || "eyJhbGciOiJIUzI1NiIsInR5cCI6I..";
const { data: subscribers } = await supabase.from("push_subscriptions").select("");
```

Impacto

Vazamento total da base de inscritos (endpoints WebPush, chaves criptográficas `p256dh/auth` e times favoritos) e potencial destruição de dados por deleção não autenticada.

Sugestão de Correção

1. Ativar Row Level Security (`ALTER TABLE push_subscriptions ENABLE ROW LEVEL SECURITY;`).
2. Criar política permitindo apenas `INSERT` para a role `anon`.
3. Utilizar a `SUPABASE_SERVICE_ROLE_KEY` exclusivamente no backend `api/cron-alerts.js` para leitura e manutenção global.

Critérios de Aceite

- RLS ativado na tabela `push_subscriptions`.
- Requisição `GET /rest/v1/push_subscriptions` com `anon key` retorna lista vazia ou erro 401/403.
- Backend utiliza chave de serviço segura para leitura de inscritos.

--- FIM ISSUE 2 ---

--- ISSUE 3 ---**Título: [Segurança] Endpoint cron-alerts desprotegido contra execução não autorizada****Labels:** security, severidade:alta, backend, api**Descrição do Problema**

O endpoint serverless `/api/cron-alerts` executa a checagem de partidas e disparo de notificações push sem verificar qualquer mecanismo de autenticação ou token de segurança.

Evidência de CódigoArquivo: `api/cron-alerts.js:20-30`

```
module.exports = async (req, res) => {  
  // Nenhuma verificação de req.headers.authorization ou CRON_SECRET  
  const { data: subscribers, error: subError } = await supabase...
```

Impacto

Ataque de negação de serviço (DoS), esgotamento rápido de requisições contratadas na API-Football e envio repetitivo/spam de notificações aos usuários.

Sugestão de Correção

1. Implementar checagem de autorização via header `Authorization: Bearer ${process.env.CRON_SECRET}` ou validar o header `x-vercel-cron` da Vercel.

Critérios de Aceite

- Chamadas HTTP sem o secret retornam 401 Unauthorized.
- Vercel Cron configurado para enviar o token de autorização.

--- FIM ISSUE 3 ---**--- ISSUE 4 ---****Título: [Segurança] Injeção de dados de notícias RSS em innerHTML sem escape (XSS)****Labels:** security, severidade:média, frontend, xss**Descrição do Problema**

A função `loadTeamNews` em `public/app.js` interpola os campos `item.title`, `item.source` e `item.link` de feeds RSS externos diretamente no innerHTML do container de notícias sem sanitização.

Evidência de CódigoArquivo: `public/app.js:1275-1288`

```
container.innerHTML = `  
<div class="news-list-compact">  
  ${newsItems.map(item => `  
    <a class="news-card-compact" href="${item.link}" target="_blank" rel="noopener noreferrer">  
      <h4 class="news-card-title">${item.title}</h4>
```

Impacto

Execução de scripts arbitrários (XSS) no contexto do navegador caso um feed RSS externo ou provedor de notícias contenha payload malicioso no título ou link.

Sugestão de Correção

1. Envolver `item.title` e `item.source` na função `escapeHtml()`.
2. Validar que `item.link` inicia estritamente com `http://` ou `https://` antes de renderizar no href.

Critérios de Aceite

- Tags HTML em títulos de notícias são renderizadas como texto plano escapado.
- Links com esquema javascript: são rejeitados.

--- FIM ISSUE 4 ---